Eliott PLANTIER

Institut Polytechnique des Sciences Avancées

Real-Time Embedded Systems Final Assignment

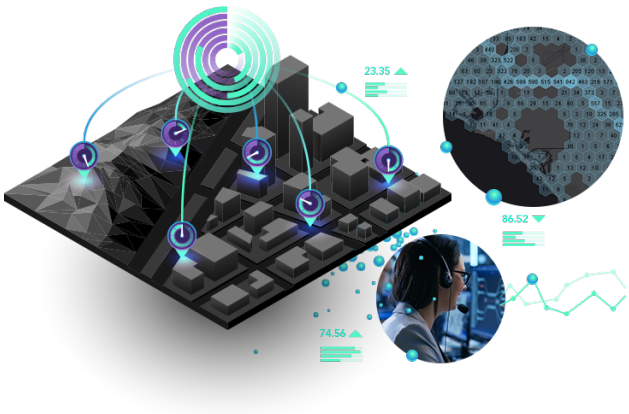

Figure 1 – Real-time information system illustration

Classe : 4TS1

Professeur : J.SINGH

Date de rendu : Tuesday 4 march 2026

Table des matières

1 Introduction **3**

2 Real-Time Model and Assumptions **4**

3 WCET Estimation of Task τ_1 **5**
3.1 Task Description 5
3.2 Measurement Method 5
3.3 Measurement Results 6
3.4 WCET Computation 6
3.5 Justification 6

4 Task Set **7**

5 Hyperperiod **7**

6 Job Generation **7**

7 Schedulability by Utilization **8**

8 Scheduling Algorithm **9**
8.1 Non-Preemptive EDF 9
8.2 Waiting Time and Response Time 9

9 Final EDF Schedule **10**

10 Response Time Verification **10**

11 Comparison of Scheduling Heuristics **11**
11.1 Earliest Deadline First 11
11.2 Shortest Job First 11
11.3 Least Laxity First 12
11.4 Comparison Table 12

12 Relaxed Constraint Case for τ_5 **12**

13 Computational Complexity **13**

14 Discussion **13**

15 Conclusion **14**

A Source Code Files **14**

1 Introduction

This report presents the final assignment for the Real-Time Embedded Systems course. The main objective is to analyze the schedulability of a periodic real-time task set and to construct a valid non-preemptive schedule at the job level.

The work carried out in this project follows a structured approach. First, task τ_1 is implemented and its execution time is experimentally evaluated. Multiple measurements are performed in order to obtain statistical indicators such as the minimum, maximum, and quartiles (Q_1, Q_2, Q_3). Based on these results, a Worst Case Execution Time (WCET), denoted C_1, is determined using a pessimistic approach.

Once C_1 is obtained, the complete task set is constructed and its schedulability is analyzed. The hyperperiod of the system is computed, allowing the generation of all jobs over a finite time interval. A non-preemptive schedule is then built, and several criteria are evaluated, including total waiting time, response times, and deadline satisfaction.

Different scheduling strategies are explored in order to identify the most suitable solution. In particular, the study focuses on minimizing the total waiting time while ensuring that all deadlines are met. The computational complexity of the scheduling algorithm is also analyzed.

Finally, an additional scenario is investigated where task τ_5 is allowed to miss a deadline. This relaxed constraint is used to study its impact on the overall system performance and to evaluate whether it leads to a better scheduling solution.

The final implementation combines a C++ program, used to accurately measure the execution time of task τ_1, and a Python-based scheduler, which automatically integrates the computed WCET and generates the complete schedule.

2 Real-Time Model and Assumptions

A real-time task is modeled as :

$$\tau_i = (C_i, T_i, D_i)$$

where :

— C_i is the worst-case execution time ;

— T_i is the period ;

— D_i is the relative deadline.

In this assignment, deadlines are assumed to be equal to periods :

$$D_i = T_i$$

This assumption is commonly used for periodic real-time task sets and is consistent with the task table given in the assignment.
The scheduler is assumed to be :

— single-core ;

— non-preemptive ;

— job-level ;

— deterministic once the job selection rule is defined ;

— based on known release times and deadlines.

Since the original assignment does not explicitly specify physical time units, all timing values are treated as consistent time units. The WCET of τ_1 is measured in seconds and then used consistently in the schedule. In practice, the numerical values are interpreted as **normalized time units** for the scheduling analysis.

3 WCET Estimation of Task τ_1

3.1 Task Description

Task τ_1 consists of repeatedly computing products of randomly generated numbers. The goal is to obtain a realistic execution time for this computation.
The C++ code used for task τ_1 is based on a fixed workload :

```cpp
long long task1() {
    volatile long long result = 0;

    for (int i = 0; i < WORKLOAD; i++) {
        long long a = rand() % 100000;
        long long b = rand() % 100000;
        result += a * b;
    }

    return result;
}
```

Listing 1 – Core of task τ_1

The keyword `volatile` is used to prevent compiler optimizations that could remove or simplify the computation. This ensures that the full workload is effectively executed, allowing for an accurate measurement of the task execution time.

3.2 Measurement Method

The execution time is measured directly inside the C++ program using a high-resolution clock. This is important because measuring the execution time externally from Python would include additional overhead such as : process creation, operating system scheduling and command execution overhead.
Therefore, using this method we measure the execution time of the actual task inside C++.
The task is executed 1000 times. For each execution, the elapsed time is recorded. Then the following statistical values are computed :

- minimum execution time ;
- first quartile Q_1 ;
- median Q_2 ;
- third quartile Q_3 ;
- maximum execution time.

3.3 Measurement Results

The following measured execution times correspond to one representative experimental run. Minor variations may occur between executions due to hardware and operating system effects ; however, the overall distribution remains consistent :

TABLE 1 – Measured execution times of task τ_1

Statistic	Value
Number of measurements	1000
Minimum	0.0024997 s
Q_1	0.0026182 s
Median Q_2	0.0026421 s
Q_3	0.0026648 s
Maximum	0.0034317 s
WCET with 20% margin	0.00401928 s

Execution time is not perfectly deterministic because it depends on the hardware, cache behavior, operating system activity, and other runtime conditions. For this reason, a pessimistic approach, of 20 %, is required in real-time systems.

3.4 WCET Computation

The WCET is computed from the maximum observed execution time with a safety margin :

$$
\begin{aligned}
C_1 = WCET = C_{\max} \times 1.2 & \quad (1)\\
= 0.0034317 \times 1.2 & \quad (2)\\
= 0.00411804 \text{ s} & \quad (3)\\
\approx 0.004 \text{ s} & \quad (4)\\
\Rightarrow \boxed{C_1 = 0.004} & \quad (5)
\end{aligned}
$$

This rounded value is intentionally pessimistic and, therefore safer for real-time analysis.

3.5 Justification

The final value is not chosen arbitrarily. It is obtained experimentally, then increased using a safety margin. This follows the real-time design principle of pessimism : the system should be validated using safe upper bounds rather than average execution times.

4 Task Set

After obtaining C_1, the task set is :

Table 2 – Final task set

Task	Execution Time C_i	Period T_i
τ_1	0.004	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

The periods and execution times are expressed in the same normalized time unit.

5 Hyperperiod

The hyperperiod is the least common multiple of all task periods :

$$H = \text{lcm}(10, 10, 20, 20, 40, 40, 80)$$

Since all periods divide 80 :

$$\boxed{H = 80}$$

The scheduling pattern is therefore analyzed over the interval : $[0, 80]$

6 Job Generation

Each periodic task generates one job every period. The number of jobs for each task over the hyperperiod is :

$$N_i = \frac{H}{T_i}$$

Table 3 – Number of jobs generated over the hyperperiod

Task	Period T_i	Number of Jobs
τ_1	10	8
τ_2	10	8
τ_3	20	4
τ_4	20	4
τ_5	40	2
τ_6	40	2
τ_7	80	1
Total	–	29

7 Schedulability by Utilization

The processor utilization is :

$$U = \sum_i \frac{C_i}{T_i}$$

Substituting the task values :

$$U = \frac{0.004}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80}$$

$$U = 0.0004 + 0.3 + 0.1 + 0.1 + 0.05 + 0.05 + 0.0375$$

$$\boxed{U = 0.638}$$

Since :

$$U < 1$$

the processor has enough capacity to execute the workload over the hyperperiod.
However, utilization alone is not sufficient to prove schedulability under a given non-preemptive scheduling policy. Therefore, an explicit schedule is computed and all response times are verified.

8 Scheduling Algorithm

8.1 Non-Preemptive EDF

The selected scheduling policy is Earliest Deadline First (EDF). At each scheduling decision time, the available job with the earliest absolute deadline is selected.
A job is available if $r_j \leq t$, where t is the current time.
The selected job is :

$$J^* = \arg \min_{J_j \in Available(t)} d_j$$

Since the schedule is non-preemptive, once a job starts, it executes until completion.

8.2 Waiting Time and Response Time

For each job :

$$W_j = s_j - r_j$$

where :

— W_j is the waiting time ;

— s_j is the start time ;

— r_j is the release time.

The response time is :

$$R_j = f_j - r_j$$

where :

— R_j is the response time ;

— f_j is the finish time.

A job meets its deadline if $f_j \leq d_j$ or $R_j \leq D_j$

9 Final EDF Schedule

Using the automatically computed WCET of τ_1, rounded to $C_1 = 0.005$, the EDF scheduler gives the following final results :

TABLE 4 – Final EDF scheduling results

Metric	Value
Total waiting time	82.12
Total idle time	21.97
Deadline misses	0

The following Gantt chart shows the final EDF schedule.

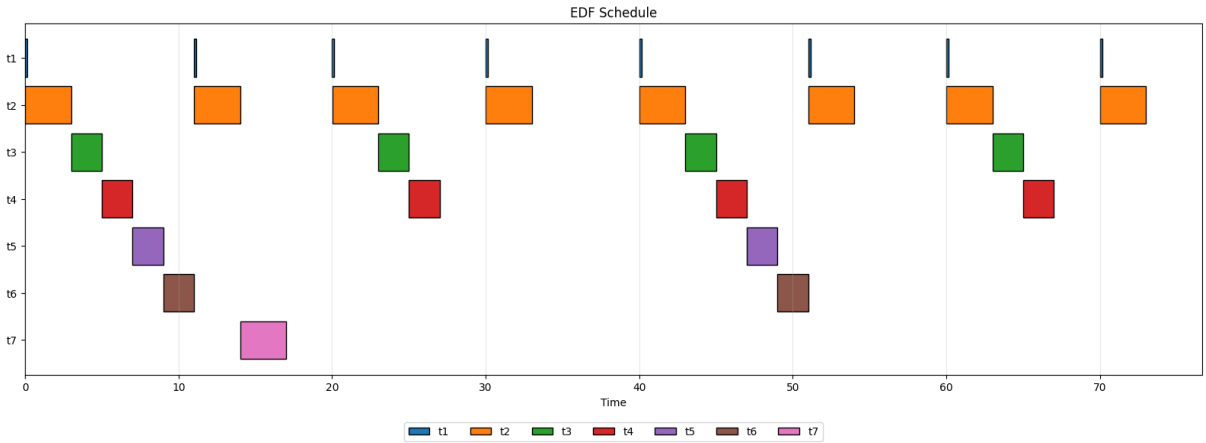

FIGURE 2 – Non-preemptive EDF schedule over the hyperperiod

Task τ_1 has a very small execution time compared with the other tasks. For readability, its visual representation in the Gantt chart is slightly enhanced while preserving the scheduling order.

10 Response Time Verification

The response time of each job is computed as $R_j = f_j - r_j$. The deadline condition is $f_j \leq d_j$.
The EDF schedule produced zero deadline misses. Therefore, all jobs satisfy : $R_j \leq D_j$. Some representative jobs are shown below.

Table 5 – Representative response-time verification

Job	Release	Start	Finish	Deadline	Deadline Met
$t1_1$	0	0.000	0.004	10	Yes
$t2_1$	0	0.005	3.005	10	Yes
$t3_1$	0	3.005	5.005	20	Yes
$t4_1$	0	5.005	7.005	20	Yes
$t5_1$	0	7.005	9.005	40	Yes
$t6_1$	0	9.005	11.005	40	Yes
$t7_1$	0	14.010	17.010	80	Yes
$t2_8$	70	70.005	73.005	80	Yes

Since no job finishes after its deadline, the schedule is feasible.

11 Comparison of Scheduling Heuristics

To evaluate the selected policy, several heuristics were tested.

11.1 Earliest Deadline First

EDF selects the job with the earliest absolute deadline.
Final result :

- total waiting time : 82.13 ;
- total idle time : 21.965 ;
- deadline misses : 0.

EDF is therefore valid.

11.2 Shortest Job First

Shortest Job First (SJF) selects the available job with the smallest execution time.
Result :

- total waiting time : 70.13 ;
- total idle time : 21.965 ;
- deadline misses : 2.

Although SJF reduces the total waiting time, it misses deadlines. The missed jobs were :

$$t2_1, \quad t2_5$$

Therefore, SJF is not acceptable for this real-time system.

11.3 Least Laxity First

Least Laxity First (LLF) selects the job with the smallest laxity :

$$L_j(t) = d_j - t - C_j$$

Result :

— total waiting time : 106.09 ;

— total idle time : 21.965 ;

— deadline misses : 0.

LLF is feasible, but it produces a higher total waiting time than EDF.

11.4 Comparison Table

Table 6 – Comparison of scheduling heuristics

Policy	Total Waiting Time	Idle Time	Deadline Misses
EDF	82.13	21.965	0
SJF	70.13	21.965	2
LLF	106.09	21.965	0

SJF gives the lowest waiting time but violates deadlines. LLF respects deadlines but increases waiting time. EDF is therefore the best valid solution among the tested heuristics.

12 Relaxed Constraint Case for τ_5

The assignment also asks to find a schedule where task τ_5 is allowed to miss a deadline. To study this case, a modified EDF heuristic was tested. The idea was to reduce the priority of τ_5 so that the other tasks are favored. The priority rule was modified as follows :

$$key(J_j) = (J_j \text{ belongs to } \tau_5,\ d_j,\ r_j)$$

This means that jobs of τ_5 are selected later when other jobs are available. The result was :

— total waiting time : 85.13 ;

— total idle time : 21.965 ;

— deadline misses : 0.

This result is worse than EDF, which gives a total waiting time of approximately 82.13. Therefore, relaxing the deadline constraint of τ_5 does not improve the final schedule for this task set. This is an important result : allowing a task to miss a deadline does not automatically improve the global performance. In this case, EDF already provides a balanced schedule.

13 Computational Complexity

Let n be the total number of jobs over the hyperperiod. At each scheduling step :

- the algorithm scans the remaining jobs to find the available jobs ;
- it selects the job with the earliest deadline among them ;
- it removes the selected job from the remaining list.

In the worst case, the algorithm scans up to n jobs for each of the n scheduling decisions. Therefore, the worst-case time complexity is : $\boxed{O(n^2)}$
The memory complexity is : $\boxed{O(n)}$ because the algorithm stores the list of jobs and the final schedule. For this assignment, $n = 29$ so the algorithm is computationally inexpensive. However, for larger real-time systems, exhaustive schedule search or advanced optimization may become much more expensive.

14 Discussion

The main difficulty of this assignment is not only to produce a schedule, but also to produce a schedule that is safe under real-time constraints. Several important observations can be made :

- The WCET must be pessimistic, not average.
- Utilization smaller than one is necessary but not sufficient.
- A schedule must be verified job by job.
- Reducing waiting time alone is not enough if deadlines are missed.
- A relaxed constraint does not necessarily improve performance.

The SJF heuristic illustrates this clearly : it improves the waiting time but misses deadlines. Since real-time systems prioritize correctness with respect to deadlines, SJF cannot be selected. EDF gives a good trade-off between waiting time and deadline satisfaction.

15 Conclusion

This project implemented and analyzed a non-preemptive real-time scheduling problem. First, task τ_1 was implemented in C++ and its WCET was measured directly inside the program. A safety margin was applied to obtain a pessimistic WCET value : $C_1 = 0.004$. Then, the full task set was generated over the hyperperiod : $H = 80$. The total number of jobs was : $n = 29$. The utilization was : $U = 0.638 < 1$ which indicates that the processor has sufficient capacity.
Several scheduling policies were tested :

— EDF ;

— SJF ;

— LLF ;

— modified EDF with relaxed τ_5 priority.

EDF was selected as the final solution because it produces :

— zero deadline misses ;

— total waiting time around 82.13 ;

— total idle time around 21.97 ;

— a simple and deterministic scheduling rule.

The response-time verification confirms that every job finishes before its deadline. Therefore, the final task set is schedulable under the EDF non-preemptive schedule.

A Source Code Files

The project is composed of the following source files :

Table 7 – Source code files

File	Description
`src/task1.cpp`	C++ implementation of task τ_1 and WCET measurement.
`src/scheduler.py`	Python scheduler using the WCET computed by the C++ program. It automatically compiles and executes `task1.cpp`, retrieves the WCET value, and generates the final EDF schedule.
`results/task1_timing_stats.txt`	Output file containing WCET measurement results.
`gantt_final_clean.png`	Gantt chart of the final EDF schedule.